

从 C++ 到计算系统：第二册

硬件原理、多核并行、高性能计算、同步、无锁结构与分布式计算

CPU Performance Study

本册接续第一册的程序执行基础，系统讲解现代 CPU 硬件、多核心并发、高性能计算、锁与无锁数据结构、并行算法、运行时和分布式计算。AI 模型、算子开发和推理引擎放入第三册。

前言

第一册回答了一个基础问题：一个 C++ 程序如何变成进程、地址空间、二进制接口、汇编指令和可测量的执行现象。第二册继续向下和向外展开，但不再提前进入 AI。真正要理解高性能程序，必须先把“计算本身”讲清楚：核心怎样取指、译码、调度和提交，乱序执行为什么能隐藏延迟，分支预测为什么会失败，缓存和 TLB 为什么决定很多程序的上限，多核心之间为什么需要一致性协议，NUMA 为什么让同一块内存存在不同核心看来成本不同。

软件层面同样不能跳步。多线程不是简单地把循环切成几份；锁不是只有“慢”一个性质；原子操作不是更快的锁；无锁数据结构也不是不用同步。并行算法要面对数据依赖、负载均衡、局部性、归约顺序和任务粒度。分布式计算更进一步：一旦机器之间只能通过网络通信，延迟、带宽、故障、重试、超时、复制和一致性就会成为计算模型的一部分。

因此第二册的主题是计算系统，而不是某个应用领域。它从硬件原理开始，讲到单机高性能计算、多线程并发、锁与无锁结构、并行算法、运行时系统、异步 I/O 和分布式计算。AI 模型、张量、算子开发和量化推理引擎会放到第三册。这样安排不是延后重点，而是先把地基打牢：没有硬件和并发基础，后续任何 AI 算子优化都会变成经验堆砌。

核心思想

第二册的核心问题是：给定真实硬件、操作系统和网络边界，怎样组织数据、线程、同步、任务和通信，使计算能够正确、可测量、可扩展地运行。

本册仍然沿用第一册的写法：原理先行，代码作为证据；实验必须记录输入、环境、命令和误差；源码阅读抓数据结构、热路径、同步边界和性能瓶颈。读者读完后，应当能解释一个多核程序为什么没有线性加速，能判断锁竞争和 false sharing 的差异，能写出可测试的并行归约和有界队列，能读懂 Linux 调度、内存和 perf 相关源码入口，也能把单机计算原则扩展到分布式系统。

怎样阅读本册

本册默认目标平台是本地 Linux 机器。普通 x86-64 Linux 台式机、笔记本或服务器适合完成完整实验；Termux 可以用于阅读、构建小实验和运行基础测试，但 perf、NUMA、线程亲和性、硬件计数器和分布式网络实验可能受权限、内核和设备能力限制。所有性能结论都必须写清环境。

阅读顺序建议严格按章节推进。前四章讲硬件执行模型，这是后面所有内容的解释基础。单机高性能计算部分讲测量、数据布局、SIMD 和典型内核，帮助读者把“程序慢”分解成计算、内存、分支、同步或 I/O 问题。并发部分讲线程、锁、条件变量、原子、内存序和无锁数据结构，重点不是背 API，而是理解共享状态怎样被保护、发布和回收。并行算法和分布式计算部分再把这些原则扩展到任务图、运行时、网络通信、复制和一致性。

本册的贯穿实验叫 Compute Systems Labs。它不追求一开始很大，而是从小而真实的模块开始：顺序归约、并行归约、带锁计数器、原子计数器、有界队列、任务切分和简单 benchmark。后续扩写会继续加入线程池、工作窃取、无锁栈、环形队列、并行扫描、分布式 shuffle 和故障注入。

心智模型

读本册时始终追问四件事：数据在哪里，谁会读写它，硬件会怎样移动它，失败或竞争发生时系统怎样保持正确。

每章至少做一个实验。硬件章节要配合 `lscpu`、`numactl`、`perfstat` 和内存访问 microbenchmark；并发章节要写出可重复触发竞争、阻塞、false sharing 或内存序问题的小程序；分布式章节即使只在本机多进程模拟，也要记录延迟、重试、超时和消息顺序。

常见误区

不要把高性能计算理解成“把线程数开大”。线程数只是资源请求，性能来自数据复用、负载均衡、同步控制和测量证据。没有这些证据，加线程常常只是更快地制造争用。

全书结构

本册分为五个部分，围绕“计算系统如何正确且高效地扩展”展开。

第一部分是硬件执行模型。它讲核心流水线、乱序执行、分支预测、寄存器重命名、缓存层级、TLB、页表、预取、NUMA、互连和缓存一致性。读者要在这里建立硬件成本模型：一条指令不只是语法，背后有取指、依赖、执行端口、访存、提交和一致性流量。

第二部分是单机高性能计算。它讲可信测量、Roofline、性能计数器、数据布局、局部性、SIMD、指令级并行和典型计算内核。目标是把“优化”从口号变成可验证的分析：到底是算力不够、内存带宽不够、cache miss 太多、分支预测差，还是 benchmark 写错了。

第三部分是多线程、同步与内存模型。它讲线程生命周期、调度、亲和性、锁、条件变量、信号量、原子操作、内存序、false sharing、无锁队列和内存回收。这里的重点是正确性先于性能：如果可见性、互斥、进度保证和对象生命周期没有说清楚，所谓高性能就是不稳定的。

第四部分是并行算法与运行时。归约、扫描、分区、图遍历、任务图、线程池、工作窃取、异步 I/O 和背压，都是把多核硬件变成可用计算能力的桥梁。读者在这里要学会从算法依赖和数据移动角度设计并行程序。

第五部分是分布式计算。它把单机原则扩展到网络和集群：RPC、序列化、时间、故障、分片、复制、一致性、共识、shuffle、检查点和观测性。分布式不是“多几台机器”，而是把通信和失败纳入计算模型。

核心思想

第二册的主线是一条从硬件到集群的链：硬件决定局部成本，多线程决定共享内存内的协作，并行算法决定可扩展形状，分布式系统决定跨机器的通信和故障边界。

目录

第一部分	硬件执行模型：计算为什么会快也会慢	
第 1 章	计算系统全景	2
第 2 章	核心流水线、乱序执行与分支预测	4
第 3 章	缓存、TLB 与虚拟内存	6
第 4 章	NUMA、互连与缓存一致性	8
第二部分	单机高性能计算：从测量到数据流	
第 5 章	可信测量、Roofline 与性能计数器	11
第 6 章	数据布局、局部性与预取	13
第 7 章	SIMD、指令级并行与向量化	15
第 8 章	典型计算内核模式	17
第三部分	多线程、同步与内存模型	
第 9 章	线程、调度与亲和性	20
第 10 章	锁、条件变量与信号量	22
第 11 章	原子操作与内存序	24
第 12 章	无锁数据结构	26
第四部分	并行算法与运行时	
第 13 章	归约、扫描与分区	29
第 14 章	任务运行时与工作窃取	30
第 15 章	I/O、异步与背压	32
第五部分	分布式计算：把单机原则扩展到集群	
第 16 章	分布式计算模型	34
第 17 章	分片、复制与共识	36
第 18 章	分布式计算工程实践	38
附录 A	实验路线	39
附录 B	源码阅读索引	40

第零部分

硬件执行模型：计算为什么会快也会慢

第 1 章

计算系统全景

第一册已经说明程序如何变成进程、虚拟地址、调用约定和机器指令。第二册继续追问：当一个程序需要利用多个核心、多个内存层级、多个线程甚至多台机器时，计算能力从哪里来，又会在哪里损失。这个问题不能只从代码出发，也不能只从硬件手册出发。真正的计算系统由硬件、操作系统、运行时、算法和通信协议共同构成。

1.1 从一条指令到一个系统

一条加法指令看起来很简单，但它并不是孤立执行的。核心要取指、译码、重命名寄存器、等待操作数、选择执行端口、写回结果并提交状态。如果操作数在寄存器里，成本很低；如果来自 L1 cache，延迟增加；如果来自 L3、远端 NUMA 节点或磁盘，程序看到的时间会跨越几个数量级。并发程序还要考虑另一个核心是否正在写同一条 cache line，分布式程序还要考虑消息是否跨越网络。

因此本册把计算系统分成五层。第一层是硬件执行，包括核心、缓存、TLB、内存控制器和互连。第二层是操作系统，包括调度、页表、系统调用、I/O 和计数器。第三层是运行时，包括线程池、任务队列、同步原语和内存分配器。第四层是算法，包括归约、扫描、分区、图遍历和负载均衡。第五层是分布式协议，包括 RPC、复制、共识、shuffle 和检查点。

核心思想

高性能不是某一层单独决定的。硬件给出能力边界，操作系统分配资源，运行时组织执行，算法决定数据依赖，分布式协议处理通信和故障。

1.2 计算成本的基本单位

分析性能时，不能只看“执行了多少行代码”。更稳定的单位是指令数、周期数、访存字节数、cache miss、TLB miss、分支误预测、同步等待时间、上下文切换、系统调用次数和网络往返延迟。不同单位之间不能随便相加，但它们共同解释程序为什么慢。

例如，一个数组求和循环的算术量很小，主要成本是把数据从内存送到核心。一个矩阵乘如果分块得当，会重复使用缓存中的数据，瓶颈可能转向浮点执行端口。一个共享队列如果每次 push 都抢同一把锁，瓶颈不是算术，也不是内存带宽，而是临界区串行化和 cache line 所有权迁移。

1.3 延迟、吞吐和并发度

系统分析经常混淆三个词：延迟、吞吐和并发度。延迟是单个操作从开始到结束的时间，吞吐是单位时间完成多少操作，并发度是同时在系统中飞行的操作数量。一个内存 load 的延迟可能很高，但如果核心能同时挂起多个 miss，整体吞吐仍然可以接受；一个网络请求延迟很高，但如果客户端保持足够多的 in-flight 请求，吞吐可以接近带宽上限。

这三个量之间有一个朴素但非常有用的关系：要填满一个延迟很高的管道，需要足够并发度。磁盘、网络、内存和 CPU 后端都可以用这个心智模型理解。区别在于，CPU 的并发度来自乱序窗口、load buffer、SIMD lane 和多核心；网络系统的并发度来自连接数、请求队列和异步 I/O；分布式计算的并发度来自分片、任务和流水线。

心智模型

延迟回答“一个操作要等多久”，吞吐回答“系统每秒能做多少”，并发度回答“为了达到吞吐，需要同时容纳多少未完成工作”。

1.4 Amdahl 和 Gustafson

多核性能讨论离不开可并行比例。Amdahl 定律说明，如果程序有一部分必须串行执行，那么线程数增加到一定程度后，加速会被串行部分限制。Gustafson 定律从另一个角度看问题：如果随着核心数增加而扩大问题规模，可并行部分也随之增加，系统仍然能获得更高总吞吐。

这两个定律不是考试公式，而是工程判断工具。一个服务端请求路径中，如果日志锁、全局分配器、单分片元数据或单线程事件循环占比很高，增加 worker 不会线性加速。一个离线计算任务如果可以把输入切成更多分片，让每个节点处理独立数据，再做树形合并，扩大规模时就更接近 Gustafson 的场景。

1.5 观察系统的三张图

学习计算系统时，建议每遇到一个程序都画三张图。第一张是数据流图：输入从哪里来，经过哪些结构，输出到哪里。第二张是执行图：哪些线程、任务或节点执行哪些阶段，哪里并行，哪里等待。第三张是成本图：每个阶段主要消耗 CPU、内存、I/O、锁等待还是网络。

这三张图可以避免把问题混成一团。一个队列积压，可能是消费者计算慢，也可能是生产者太快，也可能是下游 I/O 阻塞，也可能是队列锁竞争。只有先画清数据和执行边界，测量才知道该放在哪里。

1.6 正确性先于性能

并发和分布式计算最容易出现的错误，是把速度放在正确性之前。数据竞争可能让程序偶尔错；错误的内存序可能只在某些 CPU 上触发；无锁结构如果没有处理对象回收，可能在压力测试里才出现悬垂访问；分布式系统如果没有处理超时和重试，可能在网络抖动时重复提交请求。

本册每次讨论优化，都要先给出语义边界。共享变量由谁写，谁读，什么时候发布，什么时候回收。任务是否可以重排，结果是否依赖顺序。消息是否幂等，故障后是否可以重试。只有这些问题说清楚，性能数字才有意义。

1.7 本册贯穿实验

第二册的实验项目叫 Compute Systems Labs。第一阶段实现顺序归约、并行归约、带锁计数器、原子计数器和有界队列。后续章节会围绕它继续扩展：加入线程池、任务图、工作窃取、无锁队列、并行扫描、I/O 背压和分布式模拟。

实验一：运行 Compute Systems Labs。记录 CPU 型号、核心数、线程数、缓存层级、系统版本、编译器版本和测试命令。不要只写测试通过，还要解释每个模块分别验证了计算系统的哪一类问题。

第 2 章

核心流水线、乱序执行与分支预测

现代 CPU 核心不是按源码顺序一条一条机械执行。为了提高吞吐，核心会把指令拆成微操作，放入队列，分析依赖，尽量让互不依赖的操作并行执行。理解这一层，是理解单线程性能、SIMD 前的标量优化和并发路径的基础。

2.1 流水线不是免费并行

流水线把取指、译码、执行、访存和提交分成阶段。理想情况下，每个周期都有新指令进入，每个阶段都在工作。但真实程序有数据依赖、控制依赖和资源冲突。后一条指令需要前一条指令结果时，执行端口即使空闲也不能使用；分支目标未知时，前端可能取错指令；访存 miss 时，后端会等待数据。

流水线的关键不是“越深越快”，而是吞吐和延迟的权衡。深流水线可以提高频率，却放大分支预测失败的代价。并发程序中的短临界区也会受流水线影响：一个看似只有几条指令的锁操作，背后可能包含原子读改写、cache line 独占获取和内存序约束。

2.2 前端：取指、译码和微操作

核心前端负责把指令流喂给后端。它要从 instruction cache 取指，预测下一条指令地址，译码复杂指令，把它们转成内部微操作，并放入队列。前端跟不上时，后端即使有空闲执行端口也没有工作可做。大函数、过多分支、间接跳转、指令 cache miss 和译码压力都可能让前端成为瓶颈。

x86-64 指令长度可变，译码比定长指令集复杂。现代核心通常会使用 micro-op cache 或类似结构缓存译码结果，让热循环不用每次重新译码。这个机制解释了为什么热循环体大小、函数内联和代码布局会影响性能。内联可以减少调用开销，但过度内联会膨胀代码，伤害 instruction cache 和前端供给。

2.3 后端：执行端口和调度

后端接收微操作后，要等待操作数准备好，并把它们发射到合适执行端口。整数 ALU、浮点乘加、加载、存储、分支和地址生成都有各自资源。一个循环如果每轮需要太多 load，可能受加载端口限制；如果每轮地址计算复杂，可能受地址生成单元限制；如果依赖链很长，端口空着也不能发射。

因此分析单核性能时，要区分“端口吞吐不够”和“依赖导致发不出去”。多累加器能改善后者；SIMD 能提高每条指令处理的数据量；简化地址表达式和连续访问能减轻加载和地址生成压力。不同 CPU 微架构端口配置不同，严肃优化必须结合具体机器。

2.4 乱序执行的边界

乱序执行允许核心在不改变单线程可观察结果的前提下，提前执行已经准备好的指令。寄存器重命名消除了很多假依赖，保留站和重排序缓冲区让多个指令同时等待资源。这样做可以隐藏一部分延迟，例如在一次乘法等待时执行另一个独立加法。

但是乱序执行不能越过所有边界。真实数据依赖不能消除，cache miss 太多时可并行等待的 miss 数量有限，内存屏障会限制重排，原子操作会带来更强的顺序约束。写高性能代码时，增加独立累加器、减少循环携带依赖、让数据访问可预测，都是在帮助乱序核心找到可调度空间。

2.5 重排序缓冲区和执行窗口

乱序执行不是无限的。核心只能观察一个有限窗口内的指令。重排序缓冲区保存尚未提交的指令，load buffer 保存未完成加载，store buffer 保存尚未对外可见的存储。如果窗口被长延迟 load 占住，后续可执行工作又不够，核心会停顿。

链表遍历就是典型例子。每次读取下一个指针之前，必须等当前节点加载完成。乱序核心很难

提前知道下一步地址，所以内存级并行度很低。相比之下，遍历多个独立数组或同时处理多条链，可以让多个 load 同时飞行，更容易隐藏延迟。

2.6 Load、Store 和内存消歧

内存操作比寄存器操作复杂。核心要判断一个 load 是否可能读取前面尚未完成的 store。如果地址关系不明确，硬件会做内存消歧预测。预测错误时，需要回滚并重新执行相关操作。高性能循环中，减少指针别名、使用清晰的数组边界、避免复杂交叉读写，可以帮助硬件和编译器更大胆地调度。

store buffer 也影响并发语义。普通 store 可以先进入 store buffer，稍后再对其他核心可见。内存屏障、原子操作和锁释放会约束这种行为。C++ 内存模型的 acquire/release 最终要落到这类硬件机制上。

2.7 分支预测和控制流

分支预测让核心在条件结果出来之前先猜路径。预测正确时，流水线保持饱满；预测失败时，错误路径上的工作要被丢弃。随机分支、数据相关分支和复杂间接跳转都会破坏预测。很多高性能循环会把条件变成掩码、查表或分段处理，就是为了减少不可预测控制流。

并发程序也受分支预测影响。锁的 fast path 通常期望“不竞争”，如果竞争比例上升，分支行为 and cache 行为都会改变。无锁结构的 CAS 循环在低竞争时很短，在高竞争时会反复失败，控制流和内存系统同时恶化。

深入理解

判断一段代码是不是受分支预测影响，可以比较不同输入分布下的 cycles、branches 和 branch-misses。如果随机数据显著慢于有序数据，而内存访问量没有明显变化，分支预测通常就是主要嫌疑。

2.8 典型性能症状

前端瓶颈常表现为 IPC 低、branch miss 或 instruction cache 相关事件高，代码布局 and 热循环大小值得检查。后端执行瓶颈常表现为数据在缓存中、分支也稳定，但 IPC 仍受端口或依赖限制，适合检查 SIMD、多累加器和指令混合。内存瓶颈常表现为 cache miss、TLB miss 或带宽接近平台上限。同步瓶颈则可能表现为 IPC 低、cache line 迁移、context switch、futex 等待或原子热点。

这些症状不会自动给答案，但能排除方向。不要在内存带宽瓶颈上执着减少几条整数指令，也不要锁竞争瓶颈上只调整循环展开。性能工程的价值在于把“慢”拆成可以证伪的假设。

2.9 从硬件回到代码

核心流水线给软件三个直接启发。第一，减少长依赖链，例如把单个累加器拆成多个局部累加器。第二，减少不可预测分支，让热路径尽量直。第三，减少会强制顺序的操作，例如不必要的原子和屏障。后续章节讨论 SIMD、锁、无锁队列和并行归约时，都会回到这些原则。

实验：写两个求和版本。第一个使用单累加器，第二个使用四个独立累加器。用 `perfstat` 比较 cycles、instructions 和 IPC。再构造一个带随机分支的版本，观察 branch-misses 对时间的影响。

第 3 章

缓存、TLB 与虚拟内存

CPU 核心很快，主内存相对很慢。缓存层级的存在，是为了让核心大多数时候从更近、更小、更快的存储中取数据。虚拟内存的存在，是为了让进程拥有独立地址空间、权限保护和按需映射。高性能程序必须同时理解这两套机制。

3.1 缓存层级与 cache line

缓存不是按单个字节移动数据，而是按 cache line 移动。常见 cache line 大小是 64 字节。顺序访问数组时，一次 miss 可以带来后续多个元素；随机访问链表时，每个节点可能引入一次新的 miss。这个差异解释了为什么连续数组常常比指针结构更快。

缓存有容量、相联度和替换策略。容量不足会产生容量 miss，相联度不足会产生冲突 miss，多核心共享写会产生一致性流量。一个算法理论复杂度更低，不代表真实更快；如果它把连续访问变成随机访问，可能输给复杂度略高但局部性更好的算法。

3.2 缓存映射和冲突

缓存通常按 set associative 组织。一个地址会映射到某个 set，同一个 set 里只有有限个 way。如果多个热点地址恰好映射到同一个 set，而数量超过相联度，就会频繁互相驱逐，即使总工作集小于缓存容量也会慢。这类冲突 miss 很隐蔽，因为从容量角度看“不应该放不下”。

矩阵步长、数组对齐和结构体大小都可能触发冲突。若多个数组按相同大步长访问，并且基地址关系不佳，它们可能持续争用同一组 cache set。改变 padding、改变块大小或调整访问顺序，有时能显著改善性能。

3.3 Inclusive、Exclusive 和共享缓存

不同处理器的缓存层级策略不同。有的层级倾向 inclusive，也就是上层 cache 的内容也在下层保留；有的倾向 exclusive，尽量让不同层级存放不同内容；还有很多混合策略。软件通常不需要依赖某个具体策略，但要知道共享 LLC 不是无限资源。多个线程在同一 socket 上跑不同任务，可能互相驱逐 LLC。

这对并行任务调度有直接影响。把共享数据的任务放在共享缓存附近，可能复用 LLC；把互相干扰的大流式任务放在同一 socket，可能争抢带宽和 LLC。操作系统调度器会做一般性决策，但高性能程序常需要自己记录拓扑并做亲和性实验。

3.4 写路径和 false sharing

读缓存可以共享，写缓存必须取得 cache line 的独占所有权。两个线程写不同变量，如果这些变量落在同一条 cache line 上，硬件仍然会把它们当成同一块一致性单元反复迁移，这就是 false sharing。它不是锁竞争，却会表现出类似的扩展性下降。

解决 false sharing 的方法包括按线程分配独立槽位、使用对齐和填充、批量合并结果、减少共享写频率。并行归约通常先让每个线程写自己的 partial sum，最后再合并，正是为了避免每次加法都写同一个共享变量。

Listing 3.1 用对齐槽位减少 false sharing

```
1 struct alignas(64) LocalCounter {
2     std::int64_t value = 0;
3 };
4
5 std::vector<LocalCounter> counters(worker_count);
```



```

6 for (std::int32_t worker = 0; worker < worker_count; ++worker) {
7     counters[static_cast<std::size_t>(worker)].value += 1;
8 }

```

这段代码不是说所有结构体都要强行 64 字节对齐，而是表达一个设计原则：如果多个线程会高频写不同槽位，这些槽位不要共享同一条 cache line。否则程序表面没有共享变量，硬件层面仍然在共享一致性单元。

3.5 TLB 和页表

虚拟地址访问物理内存前需要地址翻译。TLB 缓存虚拟页到物理页的映射。工作集页数太多或访问模式太随机时，TLB miss 会增加，核心需要走页表。大页可以减少页数，提高 TLB 覆盖范围，但也会影响内存分配、NUMA 放置和碎片。

理解 TLB 对数据结构设计很重要。一个跨很多小对象的图遍历，不只会造成 cache miss，还可能造成 TLB miss。紧凑存储、池化分配和按块遍历，可以同时改善 cache 和 TLB 行为。

3.6 Page walk 的成本

TLB miss 后，硬件需要按页表层级查找映射。x86-64 常见四级或五级页表，每一级页表项本身也在内存里，虽然页表项可能被缓存，但最坏情况下一个地址翻译会触发多次内存访问。若程序随机访问大量页面，真正拖慢它的不只是数据 load，还有地址翻译。

大页把页大小从 4 KiB 扩大到 2 MiB 或更大，显著增加 TLB 覆盖范围。它适合大数组、数据库缓存和数值计算工作集，但不适合所有场景。大页可能增加内存碎片，NUMA 放置也要更谨慎。使用大页前应先用计数器确认 TLB 确实是瓶颈。

3.7 缺页和内存分配

分配内存不等于物理页已经准备好。操作系统通常按需分配页面，第一次触碰页面时可能发生 page fault。benchmark 如果把第一次触碰计入核心循环，会把页错误成本混入算法时间。严谨实验通常要预热、固定输入规模，并记录 page-faults。

缺页也分 minor fault 和 major fault。minor fault 不需要从磁盘读数据，可能只是建立页表映射；major fault 需要从存储设备读取，成本高得多。高性能计算通常要避免在热路径出现 major fault，也要避免把大量 minor fault 误判为算法本身慢。

3.8 从虚拟内存到 mmap

mmap 可以把文件或匿名内存映射到进程地址空间。它让文件 I/O 看起来像内存访问，但不意味着没有 I/O 成本。第一次访问未驻留页面时仍然会缺页，顺序扫描大文件时内核预读可能有效，随机访问时可能产生大量 page fault。

数据库、搜索引擎和日志系统常利用 mmap 或 page cache，但必须理解内核何时回收页面、何时写回脏页、何时产生 I/O 抖动。系统工程里，虚拟内存既是便利抽象，也是性能变量。

常见误区

不要把 `std::vector` 的连续性理解成所有成本都消失了。连续布局改善 cache line 和预取，但如果容量超过缓存，仍然会受内存带宽限制；如果首次访问大量新页，仍然会看到缺页成本。

实验：比较行优先访问矩阵和列优先访问矩阵。记录 cache-misses、dTLB 相关事件和时间。再把矩阵规模从 L1、L2、L3 可容纳范围逐步扩大，观察拐点。

第 4 章

NUMA、互连与缓存一致性

多核心 CPU 不是把多个核心简单放在一起。核心之间通过片上互连通信，共享某些缓存层级，连接内存控制器，并用缓存一致性协议维护共享内存语义。多 socket 机器还会引入 NUMA：不同核心访问不同内存节点的成本不同。

4.1 共享内存的硬件代价

在 C++ 程序里，多个线程似乎可以直接读写同一地址。但硬件上，写一个共享 cache line 需要让其他核心的副本失效或更新。一个原子自增不是普通加法，它需要获得 cache line 独占权，并按内存序约束发布结果。争用越激烈，cache line 在核心之间来回迁移越频繁。

这解释了为什么全局计数器扩展性差。四个线程分别做本地计数，最后合并，通常比四个线程同时 `fetch_add` 同一个原子变量快。前者减少共享写，后者让每次更新都进入一致性协议热路径。

4.2 MESI 的直觉

很多一致性协议都可以用 MESI 直觉理解：Modified 表示本核心有被修改且尚未写回的独占副本，Exclusive 表示本核心有未修改的独占副本，Shared 表示多个核心可以读，Invalid 表示副本无效。当一个核心要写 Shared 行时，需要让其他核心副本失效；当其他核心随后要读，又要重新获取数据。

软件看不到这些状态名，但能看到它们的后果。一个只读共享表可以被多个核心同时缓存，扩展性很好；一个频繁更新的共享 head 指针会让 cache line 在核心之间跳来跳去；一个包含多个线程本地字段的结构体如果没有 padding，也会因为 false sharing 触发同样的迁移。

4.3 一致性和内存模型的分工

缓存一致性解决的是“同一内存位置的多个副本如何保持一致”，内存模型解决的是“不同内存操作之间能被哪些线程以什么顺序观察”。硬件一致性不等于程序自动无数据竞争。没有同步的普通读写仍然可能在 C++ 层面形成未定义行为。

一致性协议通常围绕状态机工作，例如 Modified、Exclusive、Shared、Invalid 这类状态。软件不需要手写这些状态，但必须理解状态迁移的成本：共享读便宜，共享写贵，频繁写同一行最贵。

4.4 Store buffer 和可见性

核心执行 store 后，写入可能先进入 store buffer。对本线程来说，后续 load 可能通过 store forwarding 看到自己的写；对其他核心来说，这个写何时可见还受缓存一致性和内存序约束影响。锁释放、release store 和内存屏障会把这种可见性变成可依赖的同步关系。

这就是为什么“我在写 flag 前已经写了 data”在并发中不够。编译器和硬件都可能在允许范围内重排或延迟可见性。正确发布对象需要用 release/acquire、mutex 或其他同步机制建立 happens-before。

4.5 NUMA 放置

NUMA 机器上，内存离某些核心更近。线程在节点 A 上运行，却频繁访问节点 B 分配的内存，会产生远端访问。Linux 的 first-touch 策略让首次触碰页面的线程影响页面放置，因此并行初始化和线程绑定会影响后续计算性能。

NUMA 优化的基本原则是让计算靠近数据。大数组按块分给线程时，初始化也应按相同块并行完成。跨节点共享队列和全局分配器要谨慎，因为它们可能把远端访问和同步争用叠加在一起。

4.6 First-touch 和并行初始化

Linux 常见 first-touch 策略意味着页面通常分配到第一次写入它的线程所在 NUMA 节点。如果主线程单线程初始化一个大数组，然后工作线程分布在多个节点上处理这个数组，很多工作线程可能访问远端内存。更好的方式是让每个工作线程初始化自己后续要处理的分块。

这条规则在 benchmark 中尤其重要。若初始化方式和计算方式不一致，测到的可能是内存放置错误，而不是算法本身。实验报告必须记录初始化策略、线程绑定和 NUMA 策略。

4.7 互连不是无限带宽

核心、缓存、内存控制器和 I/O 设备之间的互连带宽有限。一个程序即使单线程局部性很好，多线程扩展后也可能撞上内存带宽或互连带宽上限。此时继续增加线程，只会让每个线程分到更少带宽，吞吐不再提高。

4.8 拓扑感知设计

拓扑感知不是过早优化，而是避免把硬件当作平面资源池。线程池可以按 NUMA 节点分组，内存池可以按节点分配，队列可以按生产者或消费者分片，归约可以先在 socket 内合并再跨 socket 合并。这样的层次化设计和分布式系统中的本地聚合很像：先减少近处流量，再把少量结果发到远处。

实验：在支持 NUMA 的机器上，用 `numactl--hardware` 查看节点。写一个大数组初始化和求和程序，比较单线程初始化、多线程初始化、线程绑定和未绑定情况下的带宽。如果没有 NUMA 机器，也要用 `lscpu` 记录拓扑并解释限制。

第零部分

单机高性能计算：从测量到数据流

第 5 章

可信测量、Roofline 与性能计数器

性能工程必须先建立测量纪律。没有输入规模、环境、编译选项、计时方式和计数器证据的数字，不能用来指导优化。错误 benchmark 经常比没有 benchmark 更危险，因为它会让人优化不存在的瓶颈。

5.1 测量对象

测量前先定义对象。是测单次操作延迟，还是测批量吞吐；是包含内存分配，还是只测热循环；是冷缓存，还是热缓存；是单线程，还是多线程；是平均值，还是尾延迟。不同对象需要不同方法，不能混用结论。

常见错误包括：把初始化时间算进内核，把编译器优化掉的循环当作结果，把首次缺页当作算法成本，把输出打印放进计时区，把不同 CPU 频率状态下的结果直接比较。

5.2 计时器和统计方法

计时器应使用单调时钟，避免系统时间调整影响。短任务要重复执行足够多次，并防止编译器消除结果。报告结果时不要只给一次运行时间，应至少给出多次运行的最小值、中位数和波动范围。最小值常代表干扰较少的情况，中位数更接近日常情况，尾部异常则提示系统噪声或资源竞争。

预热同样重要。第一次运行可能包含页错误、动态链接、分支预测冷启动、缓存冷启动和 CPU 频率爬升。若目标是冷启动延迟，就应该保留这些成本；若目标是热循环吞吐，就应该把它们排除。测量对象不同，预热策略不同。

5.3 Roofline 模型

Roofline 用算术强度连接计算峰值和内存带宽。算术强度是操作数与数据移动字节数的比值。低算术强度程序通常受内存带宽限制，高算术强度程序才可能接近计算峰值。这个模型不是精确预测器，而是帮助判断优化方向。

数组求和每读取一个整数只做一次加法，算术强度低；矩阵乘如果分块得当，每个元素可以被多次复用，算术强度高。若一个内核已经接近内存带宽上限，继续手写更复杂的算术指令可能没有意义；若内核远低于计算上限且数据复用充分，就要检查 SIMD、指令级并行和执行端口。

5.4 从字节数推导瓶颈

Roofline 的关键是先估算字节数。以 `std::int32_t` 数组求和为例，每个元素读取 4 字节，做一次整数加法。如果数组远大于缓存，主要流量来自内存读取。若机器可持续内存带宽是 40 GiB/s，那么理想情况下每秒最多读取约 100 亿个整数。实际还要扣除循环开销、预取效率、TLB 和其他系统干扰。

这个估算能提前判断优化方向。如果实测已经接近带宽上限，就不该期待换一种加法写法获得数量级提升。若实测只有带宽上限的一小部分，就要检查访问是否连续、是否触发 TLB miss、是否有分支、是否被编译器向量化、是否被线程调度干扰。

5.5 性能计数器

Linux 上常用 `perfstat` 查看 cycles、instructions、branches、branch-misses、cache-misses、page-faults、context-switches 等指标。更深入时可以用 `perfrecord` 和 `perfreport` 定位热点。不同 CPU 的事件名称不同，实验报告要记录命令和平台。

计数器要组合解释。IPC 低可能因为 cache miss，也可能因为分支失败或同步等待。cache-misses 高不一定说明程序差，流式扫描大数组本来就会不断 miss，但可能充分利用带宽。context-switches 多可能说明线程过多、阻塞频繁或系统有干扰。

5.6 实验记录模板

一份合格性能记录至少包含：机器型号、核心和 NUMA 拓扑、内存容量、编译器和编译选项、输入规模、数据类型、线程数、绑定策略、运行次数、计时范围、正确性校验、perf 命令和原始输出摘要。缺少这些信息，别人无法复现实验，你自己几周后也很难判断数字是否可靠。

性能结论要写成果因假设，而不是口号。例如“并行版本在线程数超过 8 后不再加速，perf 显示 LLC miss 和内存带宽接近平台上限，因此瓶颈更像内存带宽，而不是线程创建成本”。这样的结论可以继续被验证或推翻。

核心思想

测量不是为了得到一个数字，而是为了排除错误解释，缩小瓶颈范围，并验证优化是否真的改变了瓶颈。

实验：对 Compute Systems Labs 的顺序归约和并行归约运行 `perfstat`。记录时间、cycles、instructions、IPC、cache-misses 和 context-switches。解释并行版本是否真的更快，如果没有，瓶颈可能在哪里。

第 6 章

数据布局、局部性与预取

数据布局决定程序怎样使用缓存、TLB 和内存带宽。很多高性能优化并不是改变算法复杂度，而是把数据摆成硬件更容易消费的形状。理解布局，是理解数组、结构体、队列、图和矩阵内核的共同基础。

6.1 AoS 与 SoA

Array of Structures 把一个对象的字段放在一起，适合经常同时访问完整对象的场景。Structure of Arrays 把同类字段连续存放，适合批量处理某个字段的场景。例如粒子模拟中，如果每轮只更新所有粒子的 x 坐标，SoA 更容易形成连续访问和 SIMD；如果每次处理一个粒子的全部属性，AoS 更直观。

选择布局不能只看代码好不好写，要看热路径读写哪些字段、是否需要向量化、是否有 false sharing、是否跨页跳转。工程中常见折中是 AoSoA，把数据按小块组织，既保留局部对象关系，又适合 SIMD 和缓存块。

Listing 6.1 AoS 与 SoA 的访问差异

```
1 struct Particle {  
2     float x;  
3     float y;  
4     float z;  
5 };  
6  
7 struct ParticleSoA {  
8     std::vector<float> x;  
9     std::vector<float> y;  
10    std::vector<float> z;  
11 };
```

如果热路径只更新所有粒子的 x ，SoA 版本会连续读取和写入 x 数组；AoS 版本每次还把 y 和 z 附近的数据带入 cache line。若热路径总是同时使用 x 、 y 、 z ，AoS 的空间局部性反而更自然。

6.2 预取

硬件预取器擅长识别顺序访问和固定步长访问。连续数组扫描通常能被预取器提前加载，随机指针追踪则很难。软件预取可以在少数场景有效，但如果预取太早会挤出有用缓存，太晚则来不及隐藏延迟。

最好的预取优化通常不是手写预取指令，而是改变访问模式：把随机访问排序，按块处理，压缩节点，减少指针跳转，让硬件预取器能看懂程序。

6.3 冷热数据分离

对象里不是所有字段都同样热。把热字段和冷字段混在一起，会让缓存带入很少使用的数据。冷热分离把热路径频繁访问的字段放在紧凑结构中，把调试信息、统计字段、名字、配置等冷数据放到旁边结构或间接存储。数据库执行器、游戏引擎和网络服务都常使用这种思路。

冷热分离也有代价：代码复杂度增加，对象关系不如原来直观。是否值得做，要看热路径字段访问比例和缓存压力。设计数据结构时，先写清哪个路径是热路径，哪个字段在热路径必读或必写。

6.4 内存分配和对象布局

小对象频繁分配会带来分配器开销、碎片、TLB 压力和缓存局部性问题。高性能系统常用 arena、对象池、批量分配或结构化生命周期来减少热路径分配。并发分配器还要避免全局锁和跨线程释放导致的远端缓存流量。

常见误区

不要把“使用堆”简单等同于慢。真正的问题是生命周期是否清晰、分配是否在热路径、对象是否分散、是否跨线程释放、是否破坏局部性。

6.5 布局与并发

多线程程序的数据布局还要考虑写共享。线程本地数据应尽量分开，跨线程只读数据可以共享，跨线程写数据要减少同一 cache line 上的冲突。并行归约、线程池队列和日志缓冲区都常常需要为每个线程准备独立槽位。

6.6 结构体大小和 ABI

结构体字段顺序影响 padding 和总大小。较大的结构体会降低缓存密度，也会增加拷贝成本。公共 ABI 中随意调整字段顺序可能破坏二进制兼容，因此高性能库常把内部热结构和外部稳定接口分开。内部结构按性能布局，外部接口保持稳定语义。

实验：实现 AoS 和 SoA 两个版本的字段求和。比较顺序访问和随机访问。再构造两个线程分别写相邻字段和分离字段的场景，观察 false sharing。

第 7 章

SIMD、指令级并行与向量化

SIMD 让一条指令处理多个数据 lane，是现代 CPU 单核吞吐的重要来源。指令级并行则让多个互不依赖的操作同时在不同执行资源上推进。二者共同决定很多数值循环的上限。

7.1 从标量语义到向量语义

向量化不是把代码表面改成更复杂的形式，而是确认多个元素之间没有依赖，可以被同一条向量指令处理。数组逐元素加法容易向量化，因为每个元素独立；前缀和不容易直接向量化，因为后一个结果依赖前一个结果。

编译器自动向量化需要知道别名关系、对齐、循环边界和数据依赖。使用 `std::span` 表达连续区间有助于接口清晰，但编译器仍可能因为别名保守而放弃向量化。查看优化报告和汇编，比猜测更可靠。

7.2 别名和 restrict 思维

两个指针可能指向同一片内存时，编译器必须保守。例如 `a[i]=b[i]+c[i]` 中，如果 `a`、`b`、`c` 可能重叠，编译器要保证重叠情况下结果仍正确，这可能阻碍向量化。C++ 没有标准 `restrict` 关键字，但接口设计可以减少别名不确定性，例如使用不重叠容器、清晰文档和局部拷贝。

手写 SIMD 前，应该先让标量代码容易被编译器理解。清晰的循环边界、连续数据、少分支、少别名，是自动向量化的基本条件。

7.3 多累加器和延迟隐藏

即使不用 SIMD，多个独立累加器也能提高吞吐。单累加器形成长依赖链，每次加法依赖上一次结果；多个累加器把依赖链拆开，让乱序执行有更多可调度操作。SIMD 内核中也常见多个向量累加器，目的相同。

这就是并行归约在单线程内部也有优化空间的原因。先分成多个局部和，再合并，不仅适用于多线程，也适用于单核流水线。

Listing 7.1 多个累加器拆开依赖链

```
1 std::int64_t s0 = 0;
2 std::int64_t s1 = 0;
3 for (std::size_t i = 0; i + 1 < values.size(); i += 2) {
4     s0 += values[i];
5     s1 += values[i + 1];
6 }
7 const std::int64_t sum = s0 + s1;
```

这段代码的意义不是只用两个累加器，而是展示依赖链拆分。真实内核会根据执行端口、寄存器数量和向量宽度选择更多累加器。累加器太少隐藏不了延迟，太多会增加寄存器压力。

7.4 尾部处理和对齐

向量宽度通常不能整除数组长度，因此要处理尾部元素。常见方法是标量尾循环、掩码加载和补齐填充。对齐可以减少某些加载成本，但现代 CPU 对非对齐连续访问已经比较友好；真正要避免的是跨 cache line、跨页和随机访问。

7.5 SIMD 与线程的关系

SIMD 和多线程不是替代关系。SIMD 提高单核吞吐，多线程使用多个核心。高性能程序通常先让单线程内核足够高效，再扩展到多线程。否则多线程只会复制低效内核，并更快撞上内存带宽。

7.6 数值语义

向量化可能改变浮点运算顺序，从而改变舍入误差。整数加法在不溢出的数学模型下更容易重排，浮点加法不严格满足结合律。编译器的 `fast-math` 选项会放宽数值语义，可能提高性能，也可能改变边界行为。工程中必须明确是否允许这种变化。

实验：用编译器优化报告观察一个数组加法循环是否向量化。再写单累加器和四累加器求和，比较性能。记录编译选项和 CPU 指令集。

第 8 章

典型计算内核模式

高性能计算里反复出现一些内核模式：map、reduce、scan、stencil、矩阵块计算、直方图、排序、图遍历和队列驱动任务。掌握这些模式，比记住某个库函数更重要。

8.1 Map 与 Reduce

Map 对每个元素独立变换，通常容易并行和向量化。Reduce 把多个元素合成一个结果，存在合并顺序问题。整数求和通常结合律明确，浮点求和则会因为舍入误差导致不同合并顺序产生不同结果。

并行 reduce 的基本形态是每个线程处理一段输入，写入线程本地 partial，然后合并。这个结构减少共享写，也让每个线程顺序扫描自己的数据块。Compute Systems Labs 的 `parallel_sum` 就是这个最小模型。

8.2 Map 的融合

多个 map 连在一起时，如果每一步都写出中间数组，会增加内存流量。融合把多个逐元素变换合在一次循环里，减少中间读写。融合不是总是好事：融合后循环体变大，寄存器压力可能增加，分支也可能更复杂。是否融合，要看减少的内存流量是否超过新增的执行压力。

8.3 Stencil 与邻域计算

Stencil 计算每个输出元素时读取邻域元素，例如图像卷积、有限差分和网格模拟。它的关键是复用邻域数据，避免重复从内存加载。分块、滑动窗口和边界处理是主要问题。

8.4 矩阵和块算法

矩阵计算展示了数据复用的重要性。朴素矩阵乘的数学公式很简单，但访问顺序会决定是否复用缓存。分块算法把矩阵切成能放入缓存的小块，让一个块内的数据被多次使用。这个思想不仅用于矩阵，也用于排序、join、图处理和文件扫描。

块大小不是越大越好。太小会增加循环和调度开销，太大又放不进缓存。实际工程中常用 benchmark 或自动调优选择块大小，并针对不同 CPU 做参数化。

8.5 直方图和冲突更新

直方图看似简单：读一个元素，更新一个桶。但多个线程可能更新同一个桶，导致锁竞争或原子争用。常见优化是线程本地直方图，最后合并。这个模式体现了一个原则：把高频共享写变成低频合并。

8.6 图遍历

图遍历通常局部性差、分支多、负载不均。压缩邻接表、按层处理、前沿队列和方向优化，都是为了改善访问模式和任务分配。图算法提醒我们，不是所有问题都能靠 SIMD 和连续数组轻松解决。

8.7 排序和分区

排序是数据系统的核心内核。快速排序、归并排序、基数排序和样本排序在不同场景下表现不同。并行排序的难点不只是比较次数，还包括分区均衡、临时空间、cache 行为和跨线程合并。分布式排序还会引入 shuffle 和热点分区。

核心思想

分析计算内核时先问四个问题：每个元素读写多少字节，是否存在跨元素依赖，写入是否冲突，工作量是否均衡。

实验：在 Compute Systems Labs 中扩展一个线程本地直方图。先写全局 mutex 版本，再写线程本地版本，比较扩展性，并解释差异来自锁竞争还是 cache line 迁移。

第零部分

多线程、同步与内存模型

第 9 章

线程、调度与亲和性

线程是操作系统调度的执行实体。多线程程序能利用多核心，但也引入调度、同步、内存可见性和资源竞争。理解线程，必须同时看用户态代码和内核调度。

9.1 线程不是核心

创建八个线程不等于拥有八个物理核心。硬件可能有 SMT，一个物理核心暴露两个逻辑 CPU；系统里还运行着其他进程；线程可能被迁移到不同核心。线程迁移会破坏缓存热度，过多线程会增加上下文切换。

CPU-bound 程序通常不应远多于核心数，I/O-bound 程序可以有更多并发等待。判断依据不是经验数字，而是运行时是否在计算、等待锁、等待 I/O 或被调度器抢占。

9.2 用户线程和内核线程

C++ 的 `std::thread` 通常映射到操作系统线程。操作系统调度的是内核可见的执行实体，它拥有寄存器上下文、栈、调度状态和优先级。用户态协程则不同，协程切换通常不进入内核，它把等待点显式暴露给运行时。协程适合组织大量 I/O 等待，不会自动让 CPU 密集代码并行。

这一区分很重要。把 CPU 密集任务放进单线程协程事件循环，不会使用多个核心；把大量 I/O 等待都变成内核线程，可能造成过多栈内存、调度成本和上下文切换。运行时设计要把计算并行和等待并发分开。

9.3 调度和上下文切换

上下文切换需要保存和恢复执行状态，可能破坏 cache 和 TLB 局部性。频繁阻塞、线程过多、锁竞争和系统调用都可能增加切换。性能计数器里的 context-switches 和 migrations 可以帮助定位这类问题。

Linux CFS 调度器大体追求公平，它会根据虚拟运行时间选择任务。实时调度策略和普通调度策略不同，容器和 cgroup 也会改变 CPU 配额。benchmark 时若机器上有其他负载，或者进程受 cgroup 限制，结果会明显波动。严谨实验应记录负载、CPU governor、容器限制和是否独占机器。

9.4 阻塞、睡眠和自旋

线程等待锁时，可以自旋，也可以睡眠。自旋避免进入内核，适合等待时间极短的场景；睡眠释放 CPU，适合等待时间较长的场景。很多锁实现会先短暂自旋，再进入 futex 等内核等待路径。这样低竞争时快，高竞争时不至于烧满 CPU。

选择自旋还是阻塞，本质是在 CPU 时间和唤醒延迟之间取舍。如果临界区很短且核心充足，自旋可能好；如果锁持有者被抢占或等待 I/O，自旋会浪费大量周期。不要在没有测量的情况下把自旋当成高性能。

9.5 线程亲和性

线程亲和性让线程倾向或固定在某些 CPU 上运行。它可以改善缓存局部性和 NUMA 放置，也可能降低调度器自由度。使用亲和性前要理解拓扑：物理核心、SMT 线程、NUMA 节点和共享缓存层级。

9.6 线程生命周期

频繁创建销毁线程成本高，也让任务调度不可控。线程池把线程生命周期和任务生命周期分离，适合大量短任务。但线程池不是万能的，如果任务太小，调度成本会超过计算成本；如果任务阻塞，线程池可能被耗尽。

9.7 线程局部存储

线程局部存储让每个线程拥有独立对象，适合缓存临时缓冲、统计计数和避免共享写。它可以减少锁竞争，但也可能增加内存占用，并让跨线程汇总更复杂。线程池里使用线程局部对象时，还要注意任务迁移：同一个逻辑请求的不同阶段可能被不同 worker 执行，不能把请求状态误放到线程局部。

核心思想

线程设计的核心不是“开几个线程”，而是决定哪些状态线程本地，哪些状态共享，哪些共享状态需要同步，哪些等待应该阻塞或异步化。

实验：把并行归约的 worker 数从 1 增加到逻辑 CPU 数的两倍。记录时间、context-switches 和 migrations。解释从哪个点开始扩展性变差。

第 10 章

锁、条件变量与信号量

锁的作用是保护共享状态，让临界区在同一时间只被一个线程执行。锁不是性能敌人，也不是正确性的全部。正确使用锁，需要明确保护对象、锁顺序、临界区长度和等待条件。

10.1 互斥锁保护什么

每把锁都应该有清晰的保护范围。一个队列锁保护 head、tail、size 和 buffer 状态；一个计数器锁保护 value；一个对象锁保护对象不变量。锁如果没有对应的不变量，就容易出现“看起来加锁了但仍然错”的情况。

临界区应尽量短，但不能为了短而破坏不变量。把状态检查放在锁外，可能引入竞态；把耗时 I/O 放在锁内，会扩大阻塞范围。工程上要在正确性和粒度之间找到边界。

Listing 10.1 锁保护队列不变量

```
1 class QueueState {
2 public:
3     void push(std::int32_t value) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->values_.push_back(value);
6     }
7
8 private:
9     std::mutex mutex_;
10    std::vector<std::int32_t> values_;
11 };
```

这段代码的关键不是 `push_back` 这一行，而是锁和不变量的关系：`values_` 的内部结构不能被多个线程同时修改。如果未来再加入 `size_`、`head_` 或统计字段，它们也必须在同一把锁保护下保持一致。

10.2 锁粒度

粗粒度锁容易证明正确，但并发度低。细粒度锁提高并发度，却增加锁顺序、死锁和状态拆分复杂度。分片锁是一种常见折中：按 key、队列分段或线程组拆成多把锁，让无关操作并行，热点操作仍受保护。

锁粒度应从访问模式出发。如果所有请求都访问同一个热点 key，分片没有帮助；如果请求天然分散，分片锁可以显著减少竞争。锁优化前要先测量等待时间和竞争比例。

10.3 条件变量

条件变量用于等待某个状态变真。它必须和互斥锁配合，并在循环里检查谓词。原因是线程可能被虚假唤醒，也可能在被唤醒后发现条件已经被其他线程消费。正确模式是“持锁检查条件，不满足则等待，醒来继续检查”。

Listing 10.2 条件变量等待谓词

```
1 std::unique_lock<std::mutex> lock(this->mutex_);
2 this->not_empty_.wait(lock, [this]() {
3     return !this->values_.empty();
4 });
```

```
5 const std::int32_t value = this->values_.front();
```

这里谓词属于受锁保护的状态。通知本身不携带数据，数据在共享状态里。若不用循环检查谓词，就可能在虚假唤醒或竞争消费后读取空队列。

10.4 信号量和资源计数

信号量表示可用资源数量。它适合限制并发度、实现 bounded buffer 或控制连接池。信号量和互斥锁不同：互斥锁保护状态的一致性，信号量表达资源额度。混用时要小心锁顺序，避免死锁。

10.5 死锁和锁顺序

死锁通常需要互斥、持有并等待、不可抢占和循环等待四个条件。破坏循环等待的常见方法是规定全局锁顺序。复杂系统里，锁顺序文档和锁级别检查很重要，因为死锁往往在低概率路径中出现。

10.6 优先级反转和 convoy

优先级反转发生在低优先级线程持有锁，高优先级线程等待它，而中等优先级线程持续占用 CPU，导致高优先级线程无法前进。锁 convoy 则是多个线程排队等待同一锁，锁释放后唤醒和抢占成本让系统吞吐下降。这些问题说明锁不仅是用户态对象，也和调度器行为相互作用。

工程系统要避免在锁内做不可控工作，例如磁盘 I/O、网络调用、复杂回调和用户插件。锁内执行时间越不可预测，越容易出现尾延迟问题。

常见误区

不要把锁竞争问题简单理解成“换成无锁”。如果临界区保护的是复杂不变量，无锁实现可能更难证明正确，也可能在高竞争下 CAS 失败更多。

实验：实现一个 bounded queue。先用 mutex 和 condition variable 写阻塞 push/pop，再记录高生产者数量下的等待时间。解释锁保护的状态和等待谓词。

第 11 章

原子操作与内存序

原子操作提供不可分割的读写或读改写，并参与线程间同步。内存序定义操作之间的可见性和重排约束。理解原子操作，必须区分原子性、顺序性和生命周期。

11.1 原子不是普通变量

`std::atomic` 的 `load`、`store`、`fetch_add`、`compare_exchange` 等操作会生成特殊指令或特殊约束。它们可以避免数据竞争，但不自动保证算法正确。一个原子计数器容易写，一个无锁队列需要处理节点发布、竞争失败、ABA 和回收。

低竞争下，原子操作可能比 `mutex` 轻；高竞争下，多个线程反复修改同一 `cache line`，原子也会扩展性很差。判断依据仍然是共享写频率和一致性流量。

11.2 常见内存序

`memory_order_relaxed` 只保证原子性，不建立跨变量同步。计数器统计常用 `relaxed`，因为只关心最终数值。`memory_order_release` 发布写入，`memory_order_acquire` 获取发布的结果，二者配合可建立 `happens-before`。`memory_order_seq_cst` 提供更强的全局顺序，但成本和约束也更强。

选择内存序的原则是从语义出发，而不是从性能愿望出发。若一个线程写好对象后通过原子标志发布，写标志应 `release`，读标志应 `acquire`。若只是统计事件数量，`relaxed` 足够。

Listing 11.1 `release/acquire` 发布数据

```
1 struct Shared {
2     std::int32_t data = 0;
3     std::atomic<bool> ready = false;
4 };
5
6 void producer(Shared& shared) {
7     shared.data = 42;
8     shared.ready.store(true, std::memory_order_release);
9 }
10
11 std::int32_t consumer(Shared& shared) {
12     while (!shared.ready.load(std::memory_order_acquire)) {
13     }
14     return shared.data;
15 }
```

`release store` 之前的普通写，在 `acquire load` 看到该 `store` 后，对 `consumer` 可见。这里不能把 `ready` 简单改成普通 `bool`，也不能把 `acquire/release` 无脑改成 `relaxed`；否则程序缺少跨线程发布关系。

11.3 Relaxed 的正确用途

`Relaxed` 常用于统计计数、采样标志、引用计数的某些阶段和不参与对象发布的指标。它保证原子变量自身不会数据竞争，但不保证其他变量的可见性。用 `relaxed` 写高性能代码时，必须能说清“我只需要这个原子变量自己的数值，不需要借它同步别的数据”。

例如请求计数器可以用 `relaxed`，因为最终统计只关心总次数。对象指针发布不能只用 `relaxed`，

因为读到指针的线程还需要看到对象内部字段已经初始化。

11.4 CAS 循环

Compare-and-swap 读取旧值，若仍等于期望值就写入新值。无锁算法常用 CAS 循环处理竞争。CAS 失败不表示错误，而是说明其他线程已经修改状态，需要重新读取并尝试。

CAS 循环必须考虑进度。低竞争时很快，高竞争时可能大量失败。复杂结构还要考虑 ABA：一个位置从 A 变 B 又变回 A，CAS 看到 A 可能误以为没有变化。解决方法包括版本号、hazard pointer、epoch 回收等。

11.5 CAS 的 weak 和 strong 版本

`compare_exchange_weak` 允许伪失败，即使当前值等于期望值也可能失败，因此通常放在循环里。`compare_exchange_strong` 不允许伪失败，更适合不在循环里的单次尝试。很多平台上二者生成代码可能接近，但语义差异必须理解。

CAS 还有一个容易忽视的行为：失败时会把当前实际值写回 `expected` 参数。这让循环可以基于新值继续尝试，但也容易让初学者误用 `expected`。写无锁代码时，要把成功内存序和失败内存序都写清楚。

11.6 内存屏障和硬件

不同架构的内存模型不同。x86-64 相对强，ARM 和 POWER 更弱。C++ 内存序提供跨平台抽象，编译器和硬件会根据目标架构生成必要约束。不要因为某段代码在 x86 上“看起来没问题”，就认为它是正确的并发程序。

11.7 原子与缓存一致性

原子读改写通常会让目标 cache line 进入独占修改路径。多个核心同时对一个原子变量做 `fetch_add`，实际上是在争夺同一条 cache line 的所有权。内存序越强，编译器和硬件重排空间越小；竞争越强，一致性流量越大。

这就是为什么高性能计数常用分片计数器。每个线程或每个核心写自己的本地计数，读取总数时再合并。它牺牲实时精确读取成本，换取写入热路径扩展性。

实验：比较 mutex counter 和 atomic counter。在低线程数和高线程数下记录耗时。解释 relaxed 为什么适合这个计数器，也说明它为什么不能替代对象发布同步。

第 12 章

无锁数据结构

无锁数据结构的目标不是“没有同步”，而是在不使用互斥锁阻塞线程的情况下维护共享状态。它通常依赖原子操作、CAS、内存序和严格的对象生命周期管理。学习无锁结构，要先学习进度保证和失败模式。

12.1 进度保证

Blocking 算法可能因为一个线程持锁停顿而阻塞其他线程。Lock-free 算法保证系统整体持续前进，但单个线程可能反复失败。Wait-free 算法保证每个线程在有限步内完成，通常更难实现。Obstruction-free 只在无竞争时保证进展。

这些术语不是装饰。一个队列在压力下 CAS 反复失败，可能仍然 lock-free，但尾延迟很差。工程上需要结合竞争程度、临界区复杂度和可维护性选择方案。

12.2 无锁栈和 ABA

Treiber stack 是经典无锁栈。push 读取 head，把新节点 next 指向旧 head，再 CAS head。pop 读取 head 和 next，再 CAS head 到 next。问题在于 ABA：head 可能从 A 变成 B 又变回 A，CAS 看见 A 成功，但 A 的生命周期可能已经变化。

解决 ABA 需要额外机制。版本标签把指针和计数器一起比较；hazard pointer 让线程声明自己正在访问的节点；epoch reclamation 延迟释放节点直到所有线程离开旧 epoch。无锁算法难点往往不在 CAS 本身，而在内存回收。

12.3 内存回收为什么困难

带锁结构中，线程持有锁时可以知道没有其他线程正在修改结构。无锁结构没有这个简单边界。一个线程从 head 读到节点指针后，可能还没来得及读取 next，另一个线程已经 pop 并释放了该节点。第一个线程随后访问这个指针，就会出现 use-after-free。

垃圾回收语言可以把节点释放推迟到不可达之后，C++ 程序必须自己设计回收协议。Hazard pointer 的思路是线程公开声明“我可能访问这个节点”，释放者扫描这些声明，确认没有线程持有后再释放。Epoch 的思路是把释放延迟到所有线程都离开旧时代之后。二者都有开销，也都有使用约束。

12.4 ABA 的具体例子

假设栈顶是 A，线程 T1 读取 head=A，准备把 head 改成 A->next。此时 T2 pop A，又 pop B，然后把 A push 回栈顶。T1 再 CAS head，从 A 改成旧的 A->next。CAS 看到 head 仍然是 A，于是成功，但这个 A 已经不是 T1 当初理解的结构关系。版本标签通过让 head 变成“指针加版本号”，使 A 回来时版本不同，从而让 CAS 失败。

12.5 环形队列

有界环形队列适合固定容量、低分配、缓存友好的场景。单生产者单消费者队列可以用较简单的 head/tail 原子实现；多生产者多消费者队列需要为槽位维护序号或状态，避免多个线程同时写同一槽位。

环形队列的性能来自固定内存、连续布局 and 减少分配。但如果所有生产者争用同一个 tail，仍然会有原子热点。分片队列、每线程队列和工作窃取可以缓解热点。

12.6 SPSC、MPSC 和 MPMC

队列难度取决于生产者和消费者数量。SPSC 只有一个生产者和一个消费者，head 只被消费者写，tail 只被生产者写，可以用较轻的同步。MPSC 有多个生产者争用 tail，消费者独占 head。MPMC 两端都有竞争，通常需要更复杂的槽位序号或链表算法。

设计队列时先问清使用场景，不要直接上最复杂的 MPMC。日志线程、音频缓冲和管线阶段之间常常只需要 SPSC；线程池全局队列可能需要 MPMC；工作窃取运行时则常用每 worker deque 加窃取操作。

12.7 线性化点

并发数据结构的线性化点，是操作在逻辑上生效的那一瞬间。带锁结构的线性化点通常在锁内某次状态修改；无锁结构的线性化点通常是成功 CAS。若无法指出线性化点，就很难证明结构正确。

例如无锁栈 push 的线性化点是 CAS head 成功。CAS 失败的尝试没有生效，必须重试。测试无锁结构时，不能只看最终数量，还要考虑每个操作是否能排列成某个合法串行顺序。

12.8 何时不用无锁

如果共享状态复杂、竞争不高、临界区很短，mutex 可能更清晰也足够快。如果需要条件等待，锁加条件变量往往比忙等 CAS 更节能。无锁结构适合明确的高频热路径，且必须有严格测试、压力测试和代码审查。

实验：阅读 Compute Systems Labs 的 bounded queue，写出 mutex 版本保护的不变量。再设计一个 SPSC ring buffer 的接口，说明哪些状态可以用 relaxed，哪些位置需要 acquire/release。

第零部分

并行算法与运行时

第 13 章

归约、扫描与分区

并行算法的核心是处理依赖。归约、扫描和分区是许多系统的基础操作：数据库聚合、日志统计、图计算、排序、渲染和科学计算都会用到它们。

13.1 归约

归约把多个输入合并成一个输出。并行归约的基本策略是局部归约加全局合并。它的性能取决于输入划分、局部性、合并成本和结果类型。整数加法简单，浮点加法要考虑精度和顺序，字符串拼接则可能有大量分配。

归约优化通常先避免共享写。每个线程写自己的 `partial`，最后串行或树形合并。树形合并可以减少合并深度，也更适合分布式系统。

13.2 扫描

扫描也叫 `prefix sum`，输出每个位置之前元素的累计结果。它比归约更难，因为每个输出都依赖前缀。并行扫描通常分两阶段：先对每个块做局部扫描，得到块总和；再扫描块总和；最后把块前缀加回块内结果。

扫描是很多算法的 `building block`，例如 `stream compaction`、并行分配输出位置、基数排序和图前沿构造。理解扫描，可以帮助读者把“有顺序依赖”的问题拆成局部和全局两层。

13.3 分区

分区把元素按谓词或 `key` 分到不同区域。并行分区要先统计每个线程每个桶的数量，再计算全局偏移，最后写入输出。这与扫描密切相关。直接让多个线程 `push` 到共享 `vector`，会引入锁或原子热点。

13.4 负载均衡

静态划分适合每个元素成本相近的场景。若元素成本差异大，静态划分会让某些线程早早结束，其他线程仍在工作。动态调度、任务队列和工作窃取可以改善负载均衡，但会增加调度开销。

实验：在 `Compute Systems Labs` 中扩展 `parallel count_if`。先让每个线程本地计数，再合并。随后实现输出压缩：先统计、扫描偏移，再写输出数组。

第 14 章

任务运行时与工作窃取

线程是执行资源，任务是工作单位。把线程和任务分离，是构建可扩展运行时的关键。线程池、任务队列、工作窃取和任务图，是现代并行运行时的核心结构。

14.1 线程池

线程池创建固定数量或可控数量的工作线程，任务提交到队列，由工作线程取出执行。这样避免频繁创建线程，也便于控制并发度。线程池接口看似简单，难点在关闭、异常传播、任务等待、队列阻塞和避免死锁。

如果线程池里的任务再等待同一个线程池中的子任务，而工作线程数量不足，就可能发生饥饿。运行时设计必须考虑任务依赖，而不只是一个全局队列。

14.2 Future 和等待依赖

Future 把“将来会完成的结果”变成对象。它适合表达异步任务，但如果 worker 线程在执行任务时同步等待另一个 future，就可能形成线程池内阻塞。更好的方式是把依赖表达给运行时，让运行时在依赖满足后再调度后续任务，而不是占着 worker 等。

这也是任务图运行时的价值：等待不再消耗工作线程，依赖边决定任务何时 ready。构建系统和数据流引擎都大量依赖这种模型。

14.3 任务粒度

任务太大，负载不均；任务太小，调度成本高。合理粒度通常需要根据输入规模、核心数、缓存块和调度开销选择。递归拆分算法常设 cutoff，小于某个规模就顺序执行。

14.4 工作窃取

工作窃取让每个线程拥有本地双端队列。线程优先从本地取任务，空闲时从其他线程窃取任务。这减少全局队列争用，并改善缓存局部性。常见策略是 owner 从底部 push/pop，stealer 从顶部 steal。

工作窃取的正确性和性能都依赖队列设计。Chase-Lev deque 是经典方案，但实现复杂，涉及原子、内存序和边界竞争。教学时先理解模型，再进入源码。

14.5 本地性和窃取方向

工作窃取通常让 owner 按 LIFO 处理本地任务，让 stealer 从另一端 FIFO 窃取较老任务。LIFO 有利于缓存局部性，因为新产生的子任务往往访问相近数据；窃取较老任务则倾向拿到较大工作块，减少窃取频率。这个设计体现了运行时对局部性和负载均衡的折中。

14.6 任务图

很多计算不是简单队列，而是任务之间有依赖。任务图把节点表示任务，边表示依赖。运行时在依赖满足后调度任务。构建系统、数据处理引擎和图计算框架都大量使用任务图思想。

14.7 取消和关闭

运行时必须定义关闭语义。提交中的任务是否接受，队列里的任务是否执行完，正在运行的任务是否可取消，异常如何传播，这些都要明确。没有关闭语义的线程池很容易在程序退出时丢任务、死锁或访问已销毁对象。

实验：设计一个线程池接口，列出 submit、shutdown、wait 的语义。不要急着实现完整工作窃取，先实现固定线程数和全局队列，并写出关闭时不丢任务的测试。

第 15 章

I/O、异步与背压

计算系统不只做 CPU 运算。磁盘、网络、管道、日志和用户请求都会引入 I/O。I/O 的延迟和吞吐特征与 CPU 完全不同，因此运行时必须区分计算等待和 I/O 等待。

15.1 同步 I/O 和异步 I/O

同步 I/O 调用会让当前线程等待结果。少量线程处理少量 I/O 时简单可靠；大量并发连接或慢设备场景下，同步等待会浪费线程。异步 I/O 把提交和完成分离，让线程可以处理其他工作。

事件循环、回调、future、协程和 `io_uring` 都是组织异步 I/O 的方式。选择哪种方式，要看系统需要的并发数、延迟要求、代码复杂度和平台支持。

15.2 背压

背压是系统告诉上游“我处理不过来”的机制。没有背压，队列会无限增长，内存耗尽，延迟失控。背压可以通过 bounded queue、令牌桶、限流、拒绝请求、降级和批处理实现。

并发系统里 bounded queue 的意义不只是节省内存，更是把过载变成可观察、可控制的状态。阻塞、返回错误或丢弃，都比默默无限排队更可管理。

15.3 批处理

批处理用一次调度或一次 I/O 处理多个元素，摊薄固定成本。网络发送、日志写入、数据库提交和向量计算都常用批处理。批太小浪费固定成本，批太大增加延迟和缓存压力。

15.4 CPU 与 I/O 的边界

I/O 密集任务不应占满计算线程池。常见设计是分离 I/O 线程、计算线程和后台维护线程，或者使用异步运行时统一调度但明确资源限制。否则慢 I/O 会阻塞计算任务，计算任务也会延迟 I/O 完成处理。

实验：用 bounded queue 模拟生产者和消费者。让生产速度高于消费速度，观察队列大小和阻塞行为。解释这为什么是背压，而不是简单的性能问题。

第零部分

分布式计算：把单机原则扩展到集群

第 16 章

分布式计算模型

分布式计算把计算扩展到多台机器，但同时引入网络通信和故障。共享内存消失后，线程之间的读写变成消息；锁和原子不再直接可用；时间、顺序和失败都变得不可靠。

16.1 网络不是共享内存

单机内存访问通常以纳秒计，网络往返通常以微秒到毫秒计。单机 cache miss 已经很贵，跨网络请求更贵。分布式系统要尽量减少同步往返，使用批处理、流水线、数据本地性和异步通信。

网络还可能丢包、重排、延迟抖动或断开。应用层必须处理超时、重试、重复请求和部分失败。一个请求超时，不代表对方没有执行；重试可能造成重复副作用，因此幂等性非常重要。

16.2 RPC 的成本

RPC 把远程调用包装成类似本地函数调用的形式，但它不是本地函数。参数要序列化，消息要进入内核或用户态网络栈，经过网卡和交换机，到达对端后再反序列化和调度执行。一次 RPC 的固定成本远高于一次函数调用，因此分布式系统要避免细粒度频繁 RPC。

批处理、流水线和异步调用是常见优化。批处理减少每条消息固定开销，流水线隐藏往返延迟，异步调用避免线程阻塞。但它们都会增加错误处理复杂度：部分成功、乱序返回、超时重试和取消都要定义清楚。

16.3 时间和顺序

不同机器没有完美同步时钟。日志时间戳可以帮助观察，但不能单独作为一致性依据。分布式系统常用逻辑时钟、版本号、任期号和日志索引表达顺序关系。

16.4 幂等性

幂等操作执行一次和执行多次效果相同。分布式系统中，客户端超时后常常不知道服务器是否执行了请求。若请求不是幂等的，重试可能造成重复扣款、重复创建或重复写入。常见做法是给请求分配唯一 ID，服务器记录已处理 ID，并在重复请求时返回同一结果。

16.5 故障模型

机器可能宕机，进程可能重启，网络可能分区，磁盘可能慢，消息可能重复。多数工程系统采用 crash-stop 或 crash-recovery 模型，不处理任意恶意错误。明确故障模型，是协议设计的第一步。

16.6 数据本地性

分布式计算中，移动计算到数据附近通常比移动大量数据更划算。MapReduce、Spark、分布式数据库和对象存储都围绕数据分片、任务调度和网络 shuffle 做权衡。

16.7 CAP 的正确理解

CAP 不是说系统只能在一致性、可用性、分区容忍性里任选两个。网络分区是分布式系统必须面对的现实；在分区发生时，系统需要在继续响应请求和保持强一致之间做选择。正常情况下，系统仍然要同时追求低延迟、高可用和足够一致。工程讨论要具体到故障场景，而不是只背缩写。

核心思想

分布式系统的性能问题往往是正确性问题的影子。重试、复制、超时、共识和恢复都会改变吞吐和延迟。

实验：在本机用多个进程模拟客户端和服务端。人为加入随机延迟和超时，观察重试如何造成重复请求。设计一个幂等请求 ID 机制。

第 17 章

分片、复制与共识

分布式系统通过分片扩展容量，通过复制提高可用性，通过共识在故障中维护关键状态一致。三者共同构成大多数存储和计算平台的基础。

17.1 分片

分片把数据按 key、范围或哈希分到不同节点。好的分片要负载均衡、支持扩容、减少跨分片事务。热点 key 会破坏均衡，需要拆分、缓存、限流或特殊路由。

分片后的查询可能需要 scatter-gather：向多个分片发请求，再合并结果。它类似并行归约，但网络延迟和节点故障让合并更复杂。

17.2 再分片

系统运行一段时间后，数据量和热点会变化，需要再分片。再分片要迁移数据、更新路由、处理迁移期间的读写，并保证失败后可恢复。简单哈希分片在节点数变化时可能移动大量 key，一致性哈希或虚拟分片可以减少迁移范围。

17.3 复制

复制保存多个副本，提高可用性和读吞吐。同步复制延迟高但数据更安全，异步复制延迟低但可能丢失最新写入。主从复制简单，multi-leader 和 leaderless 复制更复杂，需要处理冲突。

17.4 读写仲裁

Quorum 系统常用 W 个写副本和 R 个读副本满足 $R+W>N$ ，从而让读集合和写集合有交集。这个模型帮助理解一致性和可用性权衡。 W 大提高写安全性但增加写延迟， R 大提高读新鲜度但增加读延迟。实际系统还要处理慢副本、hinted handoff、read repair 和版本冲突。

17.5 一致性

强一致性让读者看到最新写入，但通常需要更多同步。最终一致性允许副本暂时不同，换取可用性和低延迟。选择一致性模型要看业务语义：计费、库存和元数据通常更偏强一致；缓存、推荐和日志统计可以接受延迟收敛。

17.6 共识

Raft 和 Paxos 这类共识协议用于在故障中让多个节点就日志顺序达成一致。共识不是让所有操作都变快，而是让关键状态在故障中仍然可恢复。共识路径应尽量小而清晰，不应把所有大数据流量都压进共识日志。

17.7 Leader、任期和日志

以 Raft 的直觉看，系统在一个任期内选出 leader，客户端写入先进入 leader 日志，leader 复制给 follower，多数确认后提交。任期号防止旧 leader 在网络恢复后继续破坏新状态，日志索引和任期共同描述顺序。理解这些概念，能帮助读者读懂很多分布式存储的元数据路径。

常见误区

不要把复制等同于备份。复制会快速传播错误删除和错误写入；备份强调历史恢复。两者解决的问题不同。

实验：设计一个三副本 key-value 存储的写入流程。分别画出同步多数写、异步主从写和读修复流程，并说明各自的延迟和失败行为。

第 18 章

分布式计算工程实践

分布式计算工程把算法、运行时、存储、网络和观测性结合起来。真正的系统不仅要在正常路径跑得快，还要在机器慢、网络抖动、任务失败和数据倾斜时保持可诊断、可恢复。

18.1 Shuffle

Shuffle 把数据按 key 重新分布，是很多分布式计算框架的核心成本。它包含序列化、压缩、网络传输、落盘、排序、合并和反序列化。一个 group-by 或 join 的成本，常常主要来自 shuffle，而不是用户函数。

优化 shuffle 的方向包括减少数据量、提前聚合、压缩、分区均衡、避免热点 key、使用本地磁盘顺序写和控制并发拉取。它是分布式版的数据布局问题。

18.2 检查点和容错

长任务必须考虑失败。检查点保存中间状态，让失败后从最近点恢复，而不是从头开始。检查点太频繁会拖慢正常路径，太稀疏会增加恢复成本。流式计算还要处理 exactly-once 或 at-least-once 语义。

18.3 观测性

没有观测性，分布式系统无法维护。日志、指标、trace、profile 和事件审计要能回答：请求经过哪些节点，在哪里等待，重试了几次，队列多长，哪个分片慢，哪个任务倾斜，哪次部署引入变化。

18.4 从单机原则到集群原则

第二册前面讲的原则在分布式系统中仍然成立。局部性对应数据本地调度；锁竞争对应热点 key；false sharing 对应多个任务争用同一分片；背压对应限流和队列控制；归约对应 map-side combine 和树形聚合；缓存一致性对应复制一致性和失效策略。系统规模变大，原则没有消失，只是成本单位从 cache line 变成网络消息和磁盘块。

核心思想

分布式计算不是单机计算的替代，而是单机计算原则在通信、故障和复制约束下的扩展。

综合实验：设计一个小型分布式日志统计系统。输入按文件分片，worker 本地统计，shuffle 按 key 聚合，driver 合并结果。写清数据格式、失败重试、幂等输出、指标和瓶颈。

附录 A

实验路线

第二册的实验围绕 Compute Systems Labs 展开。第一组实验是硬件与测量基础，包括 `lscpu` 拓扑记录、缓存层级观察、TLB 与缺页计数、`perf stat`、顺序归约和并行归约。第二组实验是数据布局，包括 AoS、SoA、false sharing、顺序访问、随机访问和预取效果。第三组实验是单机高性能计算，包括多累加器、SIMD 自动向量化、Roofline 分析和典型内核模式。

第四组实验是多线程同步，包括 mutex counter、atomic counter、bounded queue、条件变量、信号量和线程亲和性。第五组实验是无锁结构，包括 SPSC ring buffer、CAS 循环、ABA 现象、版本标签和内存回收方案分析。第六组实验是并行算法，包括 reduce、scan、partition、thread-local histogram 和任务粒度比较。第七组实验是运行时，包括线程池、任务队列、工作窃取模型、异步 I/O 和背压。第八组实验是分布式计算，包括本机多进程 RPC 模拟、超时重试、幂等请求、分片、复制和 shuffle。

每个实验必须记录环境、输入、命令、输出、正确性检查和结论。性能数据没有上下文就不能进入正文。若实验受手机、Termux 或虚拟化环境限制，也要明确记录限制。

附录 B

源码阅读索引

本附录保存后续源码专题的入口。

Linux 内核源码用于理解调度、NUMA、页表、透明大页、perf 事件、futex、epoll、内存分配和网络栈。阅读时不要从全局搜索函数名开始，而要围绕问题进入：线程为什么迁移、缺页为什么发生、futex 怎样把用户态锁和内核等待连接起来。

glibc 和 musl 用于阅读 pthread、mutex、condition variable、malloc 和系统调用封装。重点观察用户态 fast path 和进入内核 slow path 的分界。

Intel TBB、oneTBB 或 LLVM OpenMP runtime 用于阅读任务调度、线程池、工作窃取和并行算法运行时。阅读时关注任务粒度、队列结构、窃取策略和 shutdown 语义。

Folly、Abseil 和 Boost.Lockfree 用于阅读并发容器、原子封装、hazard pointer、RCU 风格回收和高性能队列。阅读时先搞清进度保证和内存回收，不要只看 CAS。

Redis、RocksDB、Kafka 和 Spark 用于阅读分布式计算和存储工程：事件循环、后台线程、LSM compaction、分区、复制、日志、shuffle、背压和故障恢复。阅读时抓数据流和失败路径，不要只列模块名。

术语表

算术强度 操作数与数据移动字节数的比值，用来判断一个内核更可能受计算吞吐还是内存带宽限制。

Roofline 用计算峰值、内存带宽和算术强度估算性能上界的模型。

SIMD 一条指令处理多个数据 lane 的执行方式，是现代 CPU 单核吞吐的重要来源。

乱序执行 CPU 在保持单线程可观察结果不变的前提下，提前执行已经准备好的指令以隐藏延迟。

分支预测 CPU 在分支结果确定前预测执行路径，预测失败会清空错误路径上的工作。

cache line 缓存一致性和缓存填充的基本粒度，常见大小为 64 字节。

TLB 地址翻译缓存，用于缓存虚拟页到物理页的映射。

NUMA 非统一内存访问结构，不同核心访问不同内存节点的成本不同。

false sharing 多个线程写不同变量，但变量落在同一 cache line 上，导致一致性流量和扩展性下降。

内存序 原子操作对可见性和重排施加的约束，例如 relaxed、acquire、release 和 seq_cst。

lock-free 一类进度保证，表示系统整体能持续前进，但不保证每个线程有限步完成。

背压 下游处理不过来时向上游施加的流量控制机制。

共识 分布式系统中多个节点在故障下对日志顺序或状态达成一致的协议问题。